Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине «ОМО»

Выполнил:
Студент 3-го курса
Группы АС-68
Ярмолинский И.В.
Проверил:
Крощенко А.А.

Брест 2026

Цель работы: На практике сравнить работу нескольких алгоритмов классификации, таких как **метод k-ближайших соседей(k-NN), деревья решений** и **метод опорных векторов (SVM).** Научится подбирать гиперпараметры моделей и оценивать их влияния на результат.

Ход работы

Задачи:

1. Загрузить датасет по варианту;
2. Разделить данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке три модели: k-NN, Decision Tree и SVM;
4. Для модели k-NN исследовать, как меняется качество при разном количестве соседей (k);
5. Оценить точность каждой модели на тестовой выборке;
6. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 10

- • Adult Census Income
- • Предсказать, превышает ли доход человека $50 тыс. в год
- • Задания:

  1. Загрузите данные, обработайте пропуски и категориальные признаки;
  2. Разделите данные на обучающую и тестовую выборки;
  3. Обучите k-NN, Decision Tree и SVM;
  4. Сравните модели по метрике precision для класса ">50K";
  5. Определите, какой алгоритм лучше всего идентифицирует людей с высоким доходом.

Код программы для первого задания lab3.py:

```
import pandas as pd

import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import LinearSVC

from sklearn.metrics import precision_score

print("=" * 55)
print("ЛАБОРАТОРНАЯ РАБОТА")
```

```
print("Вариант 10 — Adult Census Income")
print("=" * 55)

url = (
    "https://archive.ics.uci.edu/"
    "ml/machine-learning-databases/adult/adult.data"
)

columns = [
    "age",
    "workclass",
    "fnlwgt",
    "education",
    "education-num",
    "marital-status",
    "occupation",
    "relationship",
    "race",
    "sex",
    "capital-gain",
    "capital-loss",
    "hours-per-week",
    "native-country",
    "income"
]

print("\nЗагрузка датасета...")

try:
    df = pd.read_csv(
        url,
        names=columns,
        na_values="?",
        skipinitialspace=True
    )
    print("Датасет успешно загружен!")

except Exception as e:
    print("Ошибка загрузки датасета:", e)
    raise SystemExit

df.to_csv("adult_orange.csv", index=False)

# Удаляем лишние пробелы
for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].str.strip()

df.to_csv("adult_orange.csv", index=False)

# Удаляем строки без целевой переменной
df = df.dropna(subset=["income"])

# Целевая переменная:
# <=50K — доход не превышает 50 тысяч
# >50K — доход превышает 50 тысяч

X = df.drop(columns=["income"])
y = df["income"]
```

```
print("\nРазмер датасета:", df.shape)

print("\nРаспределение классов:")
print(y.value_counts())

# Определяем типы признаков

numeric_features = X.select_dtypes(
    include=["int64", "float64"]
).columns.tolist()

categorical_features = X.select_dtypes(
    include=["object"]
).columns.tolist()

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nОбучающая выборка:", X_train.shape)
print("Тестовая выборка:", X_test.shape)

# Числовые признаки
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Категориальные признаки
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

# Объединяем обработчики
preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

knn = KNeighborsClassifier(
    n_neighbors=5,
    n_jobs=-1
)

tree = DecisionTreeClassifier(
    random_state=42
)

svm = LinearSVC(
    random_state=42,
    max_iter=5000
)
```

```
models = {
    "k-NN": knn,
    "Decision Tree": tree,
    "SVM": svm
}

results = []

print("\n" + "=" * 55)
print("ОБУЧЕНИЕ И ОЦЕНКА МОДЕЛЕЙ")
print("=" * 55)

for name, model in models.items():

    print("\nОбучение модели:", name)

    # Для каждой модели создаём отдельный Pipeline
    pipeline = Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("classifier", model)
    ])

    # Обучение
    pipeline.fit(X_train, y_train)

    # Предсказание
    y_pred = pipeline.predict(X_test)

    # Precision для класса >50K
    precision = precision_score(
        y_test,
        y_pred,
        pos_label=">50K",
        zero_division=0
    )

    results.append({
        "Модель": name,
        "Precision (>50K)": precision
    })

    print("Precision:", round(precision, 4))

df_result = pd.DataFrame(results)

df_result = df_result.sort_values(
    by="Precision (>50K)",
    ascending=False
)

print("\n" + "=" * 55)
print("СРАВНЕНИЕ МОДЕЛЕЙ")
print("=" * 55)

print(df_result.to_string(index=False))

best_model = df_result.iloc[0]["Модель"]
```

```
best_precision = df_result.iloc[0]["Precision (>50K)"]

print("\n" + "=" * 55)
print("ИТОГОВЫЙ РЕЗУЛЬТАТ")
print("=" * 55)

print("Лучшая модель:", best_model)
print("Precision:", round(best_precision, 4))

print("\nРабота программы завершена!")
```

Вывод результата

Загрузка датасета...
Датасет успешно загружен!
D:\OMO\lab3\lab3.py:71: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
for col in df.select_dtypes(include="object").columns:

Размер датасета: (32561, 15)

Распределение классов:
income
<=50K 24720
>50K 7841
Name: count, dtype: int64
D:\OMO\lab3\lab3.py:98: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.
categorical_features = X.select_dtypes(

Обучающая выборка: (26048, 14)
Тестовая выборка: (6513, 14)

==========
ОБУЧЕНИЕ И ОЦЕНКА МОДЕЛЕЙ
==========

Обучение модели: k-NN
Precision: 0.6775

Обучение модели: Decision Tree
Precision: 0.5993

Обучение модели: SVM
Precision: 0.7439

==========
СРАВНЕНИЕ МОДЕЛЕЙ
==========

Модель	Precision (>50K)
SVM	0.743931
k-NN	0.677465
Decision Tree	0.599284

==========
ИТОГОВЫЙ РЕЗУЛЬТАТ
==========
Лучшая модель: SVM

Precision: 0.7439

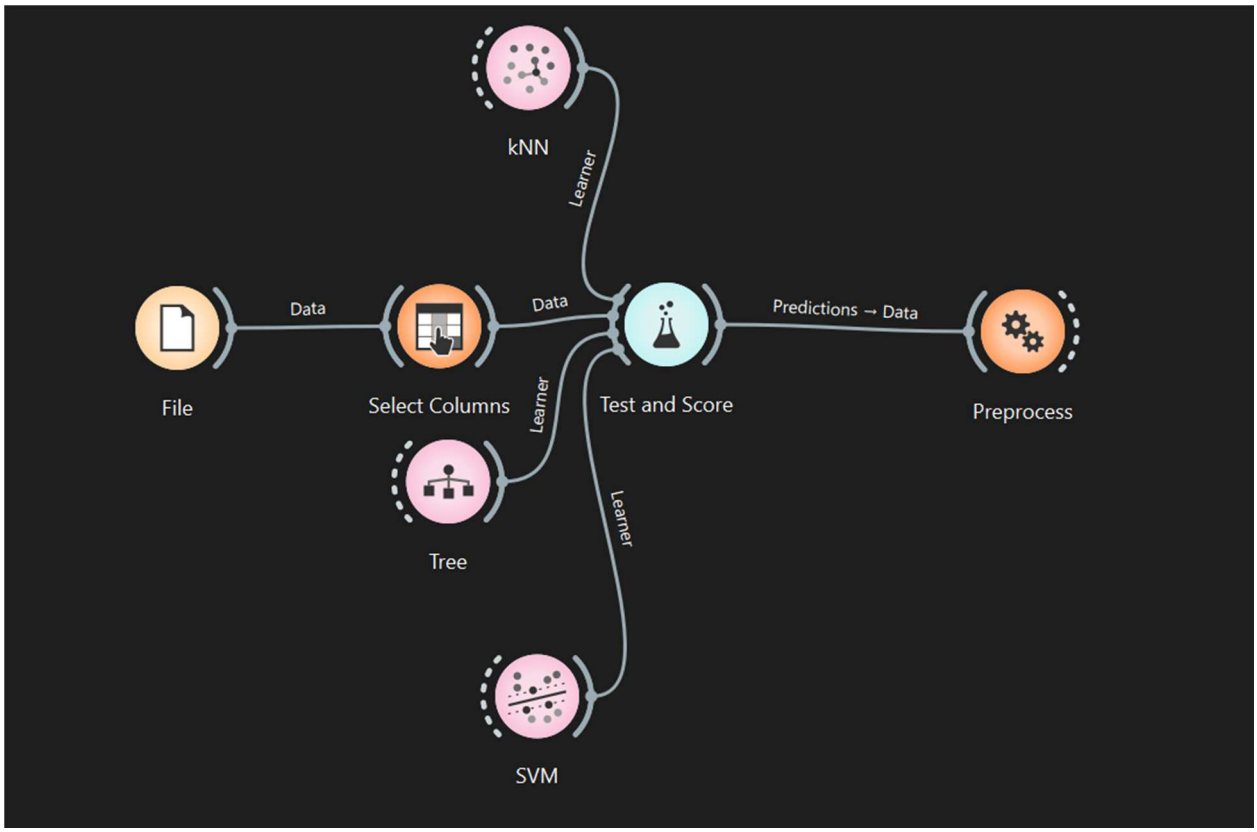
Анализ результатов на тестовой выборке показал различия в качестве работы трёх моделей машинного обучения: k-NN, Decision Tree и SVM.

Для сравнения алгоритмов использовалась метрика **Precision для класса >50K**, которая показывает, какая доля людей, отнесённых моделью к категории с доходом выше 50 тысяч долларов в год, действительно имеет такой доход.

Наиболее эффективной считается модель, показавшая наибольшее значение Precision. Она лучше остальных определяет людей с высоким годовым доходом, уменьшая количество ошибочных положительных прогнозов.

Таким образом, по результатам эксперимента можно определить наиболее подходящий алгоритм для классификации уровня дохода на основе характеристик Adult Census Income.

Результаты в Orange:



Evaluation results for target (None, show average over classes)

Model	AUC	CA	F1	Prec	Recall	MCC
Tree	0.710	0.831	0.826	0.824	0.831	0.517
SVM	0.602	0.557	0.588	0.708	0.557	0.167
kNN	0.674	0.778	0.755	0.753	0.778	0.306

Evaluation results for target <=50K

Model	AUC	CA	F1	Prec	Recall	MCC
Tree	0.706	0.831	0.891	0.872	0.910	0.517
SVM	0.648	0.557	0.640	0.835	0.519	0.167
kNN	0.674	0.778	0.863	0.811	0.922	0.306

Evaluation results for target <=50K

Model	AUC	CA	F1	Prec	Recall	MCC
Tree	0.706	0.831	0.891	0.872	0.910	0.517
SVM	0.648	0.557	0.640	0.835	0.519	0.167
kNN	0.674	0.778	0.863	0.811	0.922	0.306

Compare models by: Precision — Negligible diff.: 0.1

	Tree	SVM	kNN
Tree		0.940	1.000
SVM	0.060		0.868
kNN	0.000	0.132	

Вывод: на практике сравнил работу алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научился подбирать гиперпараметры моделей и оценивать их влияние на результат